

# Technical Report: Ticketing Platform Project

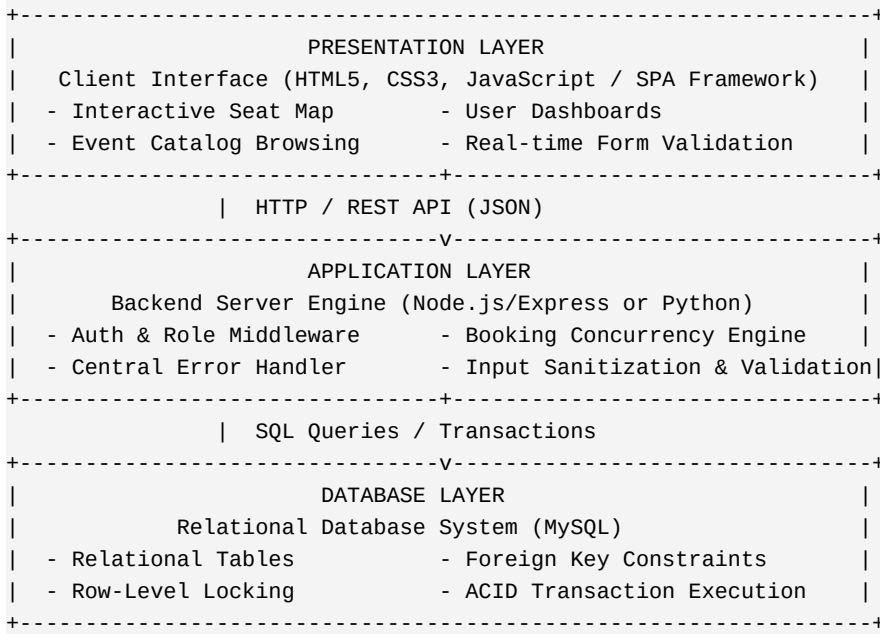
## 1. Executive Summary

This technical report provides a comprehensive overview of the Ticketing Platform Project, covering system architecture, database design, core business logic, centralized error handling mechanisms, and security guardrails.

The platform is designed as a multi-tier web application that enables users to browse events, select seats, and purchase tickets securely in real time, while granting administrators tools for event management and system auditing.

## 2. System Architecture

The platform follows a standard 3-Tier Client-Server-Database architecture to ensure separation of concerns, scalability, and maintainability.



### Architectural Component Breakdown

#### 1. Presentation Layer (Frontend):

- Responsible for rendering dynamic event listings, seat maps, and checkout workflows.
- Communicates asynchronously with the application layer via REST API calls.

#### 2. Application Layer (Backend):

- Implements authentication, role-based authorization, request routing, validation, and business logic.
- Manages stateful transactional operations such as ticket reservations and inventory decrementing.

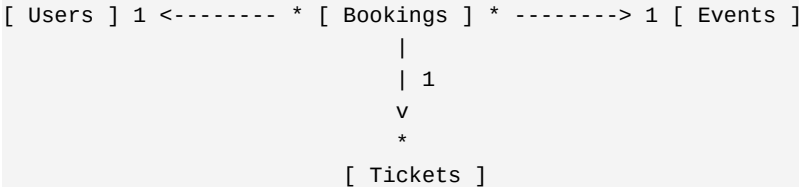
#### 3. Database Layer (Persistence):

- Relational DBMS handling persistent storage.
- Enforces referential integrity, indexes for query performance, and transactional locks for seat reservation consistency.

### 3. Database Structure & Schema Design

The relational database structure enforces data normalization (3NF) and integrity constraints to prevent orphan data and handle complex relationships between users, events, seats, and bookings.

#### Entity Relationship Overview



#### Detailed Table Specifications

**3.1 users Table** — Stores account details for both customers and platform administrators.

| Column Name   | Data Type                | Constraints                 | Description                       |
|---------------|--------------------------|-----------------------------|-----------------------------------|
| user_id       | INT                      | Primary Key, Auto Increment | Unique user identifier            |
| username      | VARCHAR(50)              | Unique, Not Null            | Unique handle for login           |
| email         | VARCHAR(100)             | Unique, Not Null            | Primary contact email             |
| password_hash | VARCHAR(255)             | Not Null                    | Cryptographically hashed password |
| role          | ENUM('customer','admin') | Default: 'customer'         | Role-based access level           |
| created_at    | TIMESTAMP                | Default: CURRENT_TIMESTAMP  | Account creation date             |

**3.2 events Table** — Stores details for events hosted on the platform.

| Column Name     | Data Type     | Constraints                            | Description                   |
|-----------------|---------------|----------------------------------------|-------------------------------|
| event_id        | INT           | Primary Key, Auto Increment            | Unique event identifier       |
| title           | VARCHAR(150)  | Not Null                               | Event name/title              |
| description     | TEXT          | Nullable                               | Event description and details |
| venue           | VARCHAR(150)  | Not Null                               | Location or venue name        |
| event_date      | DATETIME      | Not Null                               | Scheduled start date and time |
| total_seats     | INT           | Not Null, Check (total_seats >= 0)     | Total capacity                |
| available_seats | INT           | Not Null, Check (available_seats >= 0) | Remaining available tickets   |
| ticket_price    | DECIMAL(10,2) | Not Null                               | Price per ticket              |
| created_at      | TIMESTAMP     | Default: CURRENT_TIMESTAMP             | Record creation timestamp     |

**3.3 bookings Table** — Tracks customer reservation transactions.

| Column Name | Data Type | Constraints                     | Description                   |
|-------------|-----------|---------------------------------|-------------------------------|
| booking_id  | INT       | Primary Key, Auto Increment     | Unique booking transaction ID |
| user_id     | INT       | Foreign Key -> users(user_id)   | Reference to buyer            |
| event_id    | INT       | Foreign Key -> events(event_id) | Reference to target event     |
| quantity    | INT       | Not Null, Check (quantity > 0)  | Number of seats booked        |

| Column Name  | Data Type                               | Constraints                | Description                |
|--------------|-----------------------------------------|----------------------------|----------------------------|
| total_amount | DECIMAL(10,2)                           | Not Null                   | Calculated total cost      |
| status       | ENUM('confirmed','cancelled','pending') | Default: 'confirmed'       | Booking lifecycle state    |
| created_at   | TIMESTAMP                               | Default: CURRENT_TIMESTAMP | Booking creation timestamp |

**3.4 tickets Table** — Represents individual tickets issued per booking item.

| Column Name | Data Type   | Constraints                         | Description                        |
|-------------|-------------|-------------------------------------|------------------------------------|
| ticket_id   | INT         | Primary Key, Auto Increment         | Unique individual ticket ID        |
| booking_id  | INT         | Foreign Key -> bookings(booking_id) | Parent transaction ID              |
| ticket_code | VARCHAR(64) | Unique, Not Null                    | Unique barcode/hash for validation |
| seat_number | VARCHAR(20) | Not Null                            | Assigned seat identifier           |

## 4. Core Functionalities

### 4.1 Authentication & Authorization

- **User Registration & Login:** Secure authentication issuing session cookies or JSON Web Tokens (JWT).
- **Role-Based Access Control (RBAC):** Restricts administrative functions (creating/editing events) to users with the admin role.

### 4.2 Event Management & Discovery

- **Event Browsing:** Filterable event list by date, venue, and availability.
- **Admin Management:** CRUD endpoints allowing administrators to publish new events, adjust ticket prices, or update venue details.

### 4.3 Transactional Booking Engine

- **Inventory Management:** Dynamic deduction of available seats upon completed booking.
- **Atomicity:** Ensured via database transactions (BEGIN, COMMIT, ROLLBACK) to guarantee that booking records are created only if seat count decrements successfully.

### 4.4 Ticket Verification & E-Tickets

- **Unique Code Generation:** Generates unique verification hashes for each seat booked.
- **Order History:** Allows customers to review past bookings and active tickets.

## 5. Error Handling & Concurrency Control

### 5.1 Centralized Exception Architecture

All backend endpoints funnel exceptions through a unified error handler, ensuring clean error payloads and preventing application crashes.

**Standardized Error Response Payload Format (JSON):**

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_SEATS",
    "message": "The requested number of tickets is no longer available.",
    "timestamp": "2026-09-14T23:45:00Z"
  }
}
```

## 5.2 HTTP Status Code Mapping

| Scenario                | Code | Category       | Action Taken                         |
|-------------------------|------|----------------|--------------------------------------|
| Invalid form inputs     | 400  | Bad Request    | Returns input validation error array |
| Unauthenticated request | 401  | Unauthorized   | Redirects to login modal / page      |
| Access restricted route | 403  | Forbidden      | Rejects non-admin attempts           |
| Event/Booking missing   | 404  | Not Found      | Displays user-friendly 404 notice    |
| Concurrent seat claim   | 409  | Conflict       | Advises user to retry selection      |
| DB connection failure   | 500  | Internal Error | Obfuscates details, logs stack trace |

## 5.3 Concurrency & Race Condition Prevention

When multiple users attempt to purchase the last remaining ticket simultaneously:

1. **Explicit Transaction Locks:** The application executes `SELECT ... FOR UPDATE` on the event row within an isolated transaction block.
2. **Conditional Deduction:** Deductions use explicit SQL guard clauses:

```
UPDATE events
SET available_seats = available_seats - ?
WHERE event_id = ? AND available_seats >= ?;
```

3. **Rollback Execution:** If the affected row count is zero, the transaction rolls back, releasing locks and returning a 409 Conflict error to the secondary buyer.

## 5.4 Entity-Level Error Reference Matrix

The table below provides a comprehensive reference of errors organized by system entity, the corresponding HTTP status codes, and the automated system actions triggered in each case.

| Entity | Possible Error                            | HTTP Code        | System Action                                           |
|--------|-------------------------------------------|------------------|---------------------------------------------------------|
| User   | Invalid registration data                 | 400 Bad Request  | Returns validation errors and prevents account creation |
| User   | Email already exists                      | 409 Conflict     | Rejects duplicate registration request                  |
| User   | Invalid login credentials                 | 401 Unauthorized | Denies access and prompts re-authentication             |
| User   | Unauthorized access to protected resource | 403 Forbidden    | Blocks access and logs attempt                          |
| Event  | Event not found                           | 404 Not Found    | Displays user-friendly error message                    |
| Event  | Invalid event details entered by admin    | 400 Bad Request  | Rejects request and returns validation feedback         |
| Event  | Event date already passed                 | 400 Bad Request  | Prevents booking or modification                        |

| Entity   | Possible Error                        | HTTP Code                           | System Action                                          |
|----------|---------------------------------------|-------------------------------------|--------------------------------------------------------|
| Booking  | Requested tickets exceed availability | 409 Conflict                        | Stops booking and asks user to select fewer seats      |
| Booking  | Duplicate booking request             | 409 Conflict                        | Prevents duplicate transaction creation                |
| Booking  | Payment failure                       | 402 Payment Required / Custom Error | Cancels transaction and restores seat inventory        |
| Ticket   | Invalid ticket code                   | 404 Not Found                       | Rejects verification request                           |
| Ticket   | Ticket already used                   | 409 Conflict                        | Marks ticket as invalid for re-entry                   |
| Database | Foreign key constraint violation      | 400 Bad Request                     | Rejects invalid references                             |
| Database | Database connection failure           | 500 Internal Server Error           | Logs error and returns generic failure response        |
| System   | Unexpected server exception           | 500 Internal Server Error           | Centralized error handler catches and logs stack trace |

## 6. Security Specification & API Design

### 6.1 Core Security Practices

- **Password Hashing:** Passwords are never stored in plain text. Salting and hashing are enforced using bcrypt (minimum 10–12 salt rounds).
- **SQL Injection Prevention:** All SQL queries are executed using parameterized statements / prepared queries.
- **Cross-Site Scripting (XSS):** User inputs are sanitized on ingress and HTML entity-encoded prior to UI rendering.
- **CORS & Rate Limiting:** Configured middleware prevents unauthorized cross-origin API calls and mitigates brute-force attacks on auth routes.

### 6.2 Key API Endpoint Specifications

| Method | Endpoint              | Auth Required | Role     | Description                              |
|--------|-----------------------|---------------|----------|------------------------------------------|
| POST   | /api/v1/auth/register | No            | Public   | Register new user account                |
| POST   | /api/v1/auth/login    | No            | Public   | Authenticate user and obtain session/JWT |
| GET    | /api/v1/events        | No            | Public   | Fetch list of active events              |
| GET    | /api/v1/events/:id    | No            | Public   | Get detailed view of an event            |
| POST   | /api/v1/events        | Yes           | Admin    | Create a new event                       |
| POST   | /api/v1/bookings      | Yes           | Customer | Book tickets for an event                |
| GET    | /api/v1/bookings/me   | Yes           | Customer | Get current user's booking history       |

## 7. Conclusion

The Ticketing Platform architecture provides a robust, scalable foundation for ticketing services. By integrating relational database constraints, atomic transaction management, centralized error handling, and robust security protocols, the application effectively guarantees data consistency and user security under peak loading conditions.